

Software Implementation and Testing Document For Group 2

Version 3.0

Authors:

Colin Maloney

Alex Nino

Kobus VanSteenburg

Samuel Pauly

Zachary Kupersmith

1. Programming Languages (5 points)

- TypeScript – Frontend / Website (Next.JS 14)
- Python – Backend (FastAPI, ingestion pipeline, web scraping)

2. Platforms, APIs, Databases, and Other Technologies (5 points)

Frontend

- Next.JS 14 – React framework for the web application
- Vercel – Hosting and deployment (CI/CD via GitHub integration)
- Tailwind CSS – Utility-first styling; includes full dark mode support via class-based theming
- Figma – UI/UX design and prototyping

Backend

- FastAPI – REST API framework with OAuth2 token-based authentication
- SQLAlchemy – ORM for database interaction
- Alembic – Versioned database schema migrations (including composite index on TrackSnapshot)
- Python Web Scraping – Retrieves Spotify stream counts to replace deprecated popularity score
- PyTest – Testing framework for unit and integration tests

APIs

- Spotify Web API – Track and artist metadata enrichment
- TikTok API (Unofficial) – Trending sound discovery via hashtag ingestion

Database & Infrastructure

- PostgreSQL 15 – Primary data store (hosted on AWS RDS)
- QuickDBDiagrams – Database schema design and documentation
- GitHub Actions – CI/CD pipeline; enforces Python PEP-8 linting on all pull requests

3. Execution-based Functional Testing (10 points)

Test cases were written for individual components of the backend workflow using mocked API responses to avoid exhausting rate-limited endpoints. The PyTest framework is used to run all tests and produce a detailed pass/fail breakdown.

Backend Tests

- **Ingestion Pipeline:** Tests verify that TikTok sound metadata is correctly extracted, deduplicated by sound ID, and passed to the Spotify enrichment step. Mocked TikTok API responses are used to simulate trending hashtag results.
- **TrackSnapshot Creation:** Tests confirm that a TrackSnapshot record (containing stream count and timestamp) is written to the database on every ingestion run. Verifies that time-series data is being accumulated correctly.
- **Stream Scraping:** Tests validate that the web scraping module correctly extracts stream counts from Spotify track pages and returns a parseable numeric value. Mocked HTML responses are used.
- **get_library_stats Query:** Tests confirm the library stats endpoint returns the correct total track count and the correct sum of the most recent snapshot stream counts across all tracked songs, fixing the previously inaccurate homepage statistics.
- **Track Ranking Logic:** Tests verify that tracks are ranked and sorted correctly by stream momentum, ensuring the top-growing tracks are ordered as expected.
- **Duplicate Prevention:** Tests confirm that re-ingesting a previously seen Spotify track ID does not create a duplicate track or artist record in the database.
- **Error Handling:** Tests verify that failed Spotify or TikTok API calls are caught gracefully and that pipeline execution continues without crashing.

Frontend Tests

- **Charts Page:** Validates that the Charts component renders all tracked songs, correctly filters results based on search input, and applies sorting logic.
- **Track Modal:** Confirms that clicking a track card opens the expanded modal and that the stream-over-time line graph renders with the correct data points.
- **Home Page Stats:** Verifies that the homepage displays the true total tracks and total stream count sourced from the get_library_stats API endpoint.
- **Album Artwork:** Verifies that album cover images are fetched from the image_url field and rendered correctly on track cards on both the home and charts pages.
- **Search Bar:** Tests that the search bar on the Charts page queries the database and returns matching results, and that the component handles empty results gracefully.

4. Execution-based Non-Functional Testing (10 points)

- **Structured Logging:** The Python logging module is configured across all pipeline stages (ingestion, enrichment, snapshot creation, error handling). Logs are validated to confirm correct stage labels and severity levels are emitted at runtime.
- **Database Connection Resilience:** Connection pool pre-ping is enabled on the SQLAlchemy engine. Tests simulate a stale connection scenario and confirm the pool recovers without requiring a manual restart.

- **Composite Index Performance:** An Alembic migration adds a composite index on TrackSnapshot(track_id, recorded_at). Query plans were inspected via EXPLAIN ANALYZE to confirm the index is used for time-series lookups, reducing full-table scans.
- **Environment Variable Security:** A .env.example file is provided with placeholder values. A grep scan of the codebase confirms no Spotify credentials, database connection strings, or other secrets are hardcoded. Validated pre-merge via GitHub Actions.

5. Non-Execution-based Testing (10 points)

- **Code Review via Pull Requests:** All changes are submitted through pull requests and reviewed by at least one other team member before merging into increment-3. This applies to both frontend and backend contributions.
- **PEP-8 Linting (GitHub Actions):** GitHub Workflow CI is configured to run a Python linter on every push and pull request. PRs that fail the PEP-8 style check are blocked from merging.